Høgskulen på Vestlandet

DAT107 - Databaser

Video 17: MongoDB – Datamodelling

Introduksjon til NoSQL

Dag Larsen
Oppdatert av Lasse Jenssen

Læringsmål

- Hvorfor er **skjemadesign** viktig?
- Relasjonel vs. MongoDB skjemadesign
- Ulike typer for av **relasjoner** (litt repetisjon)
- MongoDB & **Patterns**

Litteratur

NoSQL

- ✓ Foiler
- ✓ https://www.mongodb.com/resources/products/fundamentals/embedded-mongodb (Embedding vs Referencing)
- ✓ https://www.mongodb.com/docs/v8.0/data-modeling/best-practices/ (Best Practices)

Høgskulen på Vestlandet

Hvorfor er skjemadesign viktig?

MongoDB Data Modellering og Database Design

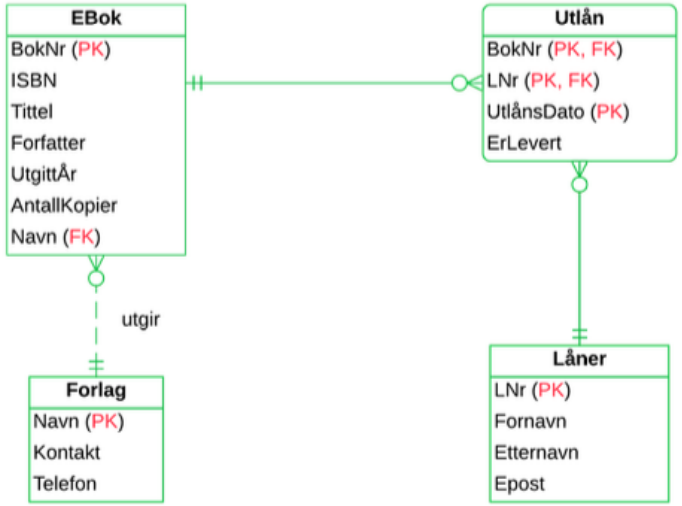
Høgskulen på Vestlandet

- ✓ Dataintegritet
- ✓ Effektivitet og ytelse
- ✓ Skalerbarhet
- ✓ Bruker og utvikler vennlighet
- ✓ Vedlikehold

Relasjonsverden

Skjema design

Utvikle applikasjon



MongoDB

Utvikle applikasjon

Definisjon av datamodell

Forbedre applikasjon

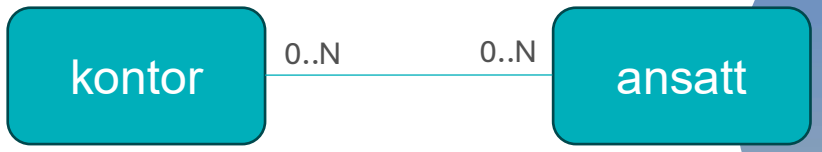
Forbedre datamo

Relasjoner i MongoDB

MongoDB Data Modellering og Database Design

Høgskulen på Vestlandet

- ✓ En-til-en relasjoner
- ✓ En-til-FÅ relasjoner
- ✓ En-til-mange relasjoner
- ✓ En-til-VELDIG-mange relasjoner
- ✓ Mange-til-mange relasjoner



Linking vs. Embedding

Hva betyr dette? Og hva skal vi velge?

Høgskulen på Vestlandet

Linking *(også kalt «referencing»)*

En-til-en relasjon

Høgskulen på Vestlandet

Ansatt

```
{
    _id: 1,
    fornavn: "Ola",
    etternavn: "Hansen",
    epost: "ola.hansen@hvl.no",
    kontor: 10
}
```

Kontor

```
{
    _id: 10,
    rom: "401",
    etasje: 4
    fløy: "A»,
    ansatt: 1
}
```

Her så har man lagt inn relasjonen begge veier.
Dette trenger man nødvendigvis ikke (det avhenger av bruken).

Embedding *(på norsk «innebygging»)*

En-til-en relasjon

Høgskulen på Vestlandet

Ansatt

```
{
    _id: 1,
    fornavn: "Ola",
    etternavn: "Hansen",
    epost: "ola.hansen@hvl.no",
    kontor: {rom: "401", etasje: 4, fløy: "A"}
}
```

Kontor

Her må man lese dokument «Ansatt» for å få tak i informasjon om kontorer.
Når man henter informasjon om ansatt får man også informasjon om kontor.

Embedding *(på norsk «innebygging»)*

En-til-en relasjon

Høgskulen på Vestlandet

Ansatt

```
{
    _id: 1,
    fornavn: "Ola",
    etternavn: "Hansen",
    epost: "ola.hansen@hvl.no",
    kontor: {rom: "401", etasje: 4, fløy: "A"}
}
```

Kontor

```
{
    _id: 10,
    rom: "401",
    etasje: 4
    fløy: "A",
    ansatt: {
        fornavn: "Ola", etternavn: "Hansen",
        epost: " ola.hansen@hvl.no"
    }
}
```

Hva legger dere merke til her?

Linking

En-til-mange relasjon: Et kontor kan ha mange ansatte

Høgskulen på Vestlandet

Ansatt

```
{
    _id: 1,
    fornavn: "Ola",
    ...
    kontor: 10
}

{
    _id: 2,
    fornavn: "Kari",
    ...
    kontor: 10
}
```

Skalære verdier

Kontor

```
{
    _id: 10,
    rom: "401",
    etasje: 4
    fløy: "A",
    ansatte: [1,2,3]
}
```

Array av verdier

Hvilken retning vi velger kan variere

En-til-mange relasjon med «linking»: Et kontor kan ha mange ansatte

Høgskulen på Vestlandet

Ansatt

```
{
    _id: 1,
    fornavn: "Ola",
    ...
    kontor: 10
}

{
    _id: 2,
    fornavn: "Kari",
    ...
    kontor: 10
}
```

Skalære verdier

Kontor

```
{
    _id: 10,
    rom: "401",
    etasje: 4
    fløy: "A",
    ansatte: [1,2]
}
```

Array av verdier

Embedding

En-til-mange relasjon: Et kontor kan ha mange ansatte

Høgskulen på Vestlandet

Ansatt

```
{
    _id: 1,
    fornavn: "Ola",
    ...
    kontor: {rom: "401", etasje: 4, fløy: "A"}
}
{
    _id: 2,
    fornavn: "Kari",
    ...
    kontor: {rom: "401", etasje: 4, fløy: "A"}
}
```

Kontor

```
{
   _id: 10,
   rom: "401",
   etasje: 4
   fløy: "A»,
   ansatte: [
    {fornavn: "Ola", etternavn: "Hansen", ... },
    {fornavn: "Kari", etternavn: "Monsen", ...}
  ]
}
```

Kan gå begge veier, eller en av veiene som ved linking.

Embedding: En-til-FÅ

En-til-mange relasjon: Et kontor kan ha mange ansatte

Høgskulen på Vestlandet

Regel 1:
Foretrekk «embedding» med mindre det er en «tvingende» grunn til å la være

Regel 2:
Behov for direkte tilgang til et objekt alene er en «tvingende» grunn for å ikke benytte «embedding»

Regel 3:
Unngå JOINs and $lookups hvis du kan, men ikke hver redd for å bruke «linking» hvis det gir et bedre schema design

Kontor

```
{
   _id: 10,
   rom: "401",
   etasje: 4
   fløy: "A»,
   ansatte: [
    {fornavn: "Ola", etternavn: "Hansen", ... },
    {fornavn: "Kari", etternavn: "Monsen", ...}
  ]
}
```

Best practise! Og ikke vitenskapelige teorier.

Linking: En-til-VELDIG-mange

En-til-mange relasjon: En maskin kan ha mange log innslag

Høgskulen på Vestlandet

Maskin

```
{
   _id: ObjectId("AAA"),
   navn: "neptun.hvl.no",
   ipaddr: "196.1.168.10»,
   log: [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,
         17,18,19,20,21,22,23,24,25,26,27,28,
         29,30,31,32,33,34,35,36,37,38,39,40,
         41,42,43,44,45,46,47,48,49,50,51,52,
         53,54,55,56,57,58,59,60,61,62,63,
         65,66,67,68,69,70,71,72,73,74,75,76,
         77,78,79,80,81,82,83,84,85,86,87,88,
         89,90,91,92,93,94,95,96,97,98,99,100,
         101,102,103,104,105, 106, 107, 108, ...
   ]
}
```

Log

```
{
   _id: 1
   time: IsoDate("2025-03-01T00:01:23.123"),
   msg: "CPU is on fire"

   _id: 2
   time: IsoDate("2025-03-01T00:05:03.036"),
   msg: "Disk is 100% full"
}
...
```

Linking: En-til-VELDIG-mange

En-til-mange relasjon: En maskin kan ha mange log innslag

Høgskulen på Vestlandet

Maskin

```
{
    _id: ObjectId("AAA"),
    navn: "neptun.hvl.no",
    ipaddr: "196.1.168.10»,
}
```

Log

```
{
    _id: 1
    time: IsoDate("2025-03-01T00:01:23.123"),
    msg: "CPU is on fire",
    host: ObjectId("AAA")
}

{
    _id: 2
    time: IsoDate("2025-03-01T00:05:03.036"),
    msg: "Drive is 100% full",
    host: ObjectId("AAA")
}
...
```

Linking: En-til-VELDIG-mange

En-til-mange relasjon: En maskin kan ha mange log innslag

Høgskulen på Vestlandet

Maskin
{
_id: ectId("AAA"),
navn: "n.hvl.no",
ipaddr: "19 68.10»,
log: [1,2,3,4,5,6, 11,12,13,14,15,16,
17,18,19,20 3,24,25,26,27,28,
29,30,31 ,34, 6,37,38,39,40,
41,42 ,45,46,47, 9,50,51,52,
53 ,56,57,58,59,60, 2,63,64,
,67,68,69,70,71,72,73, 5,76,
,78,79,80,81,82,83,84,85,86, ,88,
89,90,91,92,93,94,95,96,97,98,99,100,
101,102,103,104,105, 106, 107, 108, ...
]
}

Regel 4:
Arrays bør **ikke** få vokse uten noen form for grenser.

Kombinasjon: Linking & Embedding

En-til-mange relasjon: Et kontor kan ha flere ansatte

Ansatt

```
{
    _id: 1,
    fornavn: "Ola",
    ...
    kontor: 10
}
{
    _id: 2,
    fornavn: "Kari",
    ...
    kontor: 10
}
```

Kontor

```
{
    _id: 10,
    rom: "401",
    etasje: 4
    fløy: "A",
    ansatte: [
     {fornavn: "Ola", etternavn: "Hansen", ... },
     {fornavn: "Kari", etternavn: "Monsen", ...}
   ]
}
```

Spørsmål:
Ville dere benyttet «linking» eller «embedding» på «datter» siden?

Hva med «mor» siden?

Linking

Mange-til-mange: Et kontor kan ha flere ansatte, og en ansatt kan ha flere kontorer

Ansatte

```
{
    _id: 1,
    fornavn: "Ola",
    ...
    kontor: [10,20]
}

{
    _id: 2,
    fornavn: "Kari",
    ...
    kontor: [10]
}
```

Kontor

```
{
    _id: 10,
    rom: "401",  etasje: 4,  fløy: "A",
    ansatte: [1,2]
}
{
    _id: 20,
    rom: "12B",  etasje: 1,  fløy: "B",
    ansatte: [1]
}
```

Embedding

Mange-til-mange: Et kontor kan ha flere ansatte, og en ansatt kan ha flere kontorer

Ansatt

```
{
    _id: 1,  fornavn: "Ola", ...
    kontor: [ {rom: "401", etasje: 4, fløy: "A"},
              {rom: "12B", etasje: 1, fløy: "B"}]
}
{
    _id: 2, fornavn: "Kari", ...
    kontor: [{rom: "401", etasje: 4, fløy: "A"}]
}
```

Kontor

```
{
    _id: 10, rom: "401",  etasje: 4,  fløy: "A",
    ansatte: [{fornavn: "Ola",  ... },
              {fornavn: "Kari", ... }]
}
{
    _id: 20, rom: "12B",  etasje: 1,  fløy: "B",
    ansatte: [{fornavn: "Ola", ... }]
}
```

Er det ikke fremdeles en mange-til-mange relasjon?

Embedding

Fordeler & Ukemper

Fordeler	Ulemper
Lar oss hente ut data med en enkelt spørring	Store dokumenter === overhead (ofte trenger applikasjon bare deler av dokumentet)
Unngå dyre JOINs eller $lookup	16 MB Document size limit
Oppdate data med en enkelt **atomisk** operasjon	

Høgskulen på Vestlandet

Linking

Fordeler & Ukemper

Fordeler	Ulemper
Mindre dokumenter	To spørringer eller $lookup kreves for å hente data
Mindre sjanse for å nå 16 MB begrensning	
Ingen duplisering av data	
Unngår at man laster ned data som ikke trengs unødvendig	

Høgskulen på Vestlandet

Linking

Mange-til-mange: Et kontor kan ha flere ansatte, og en ansatt kan ha flere kontorer

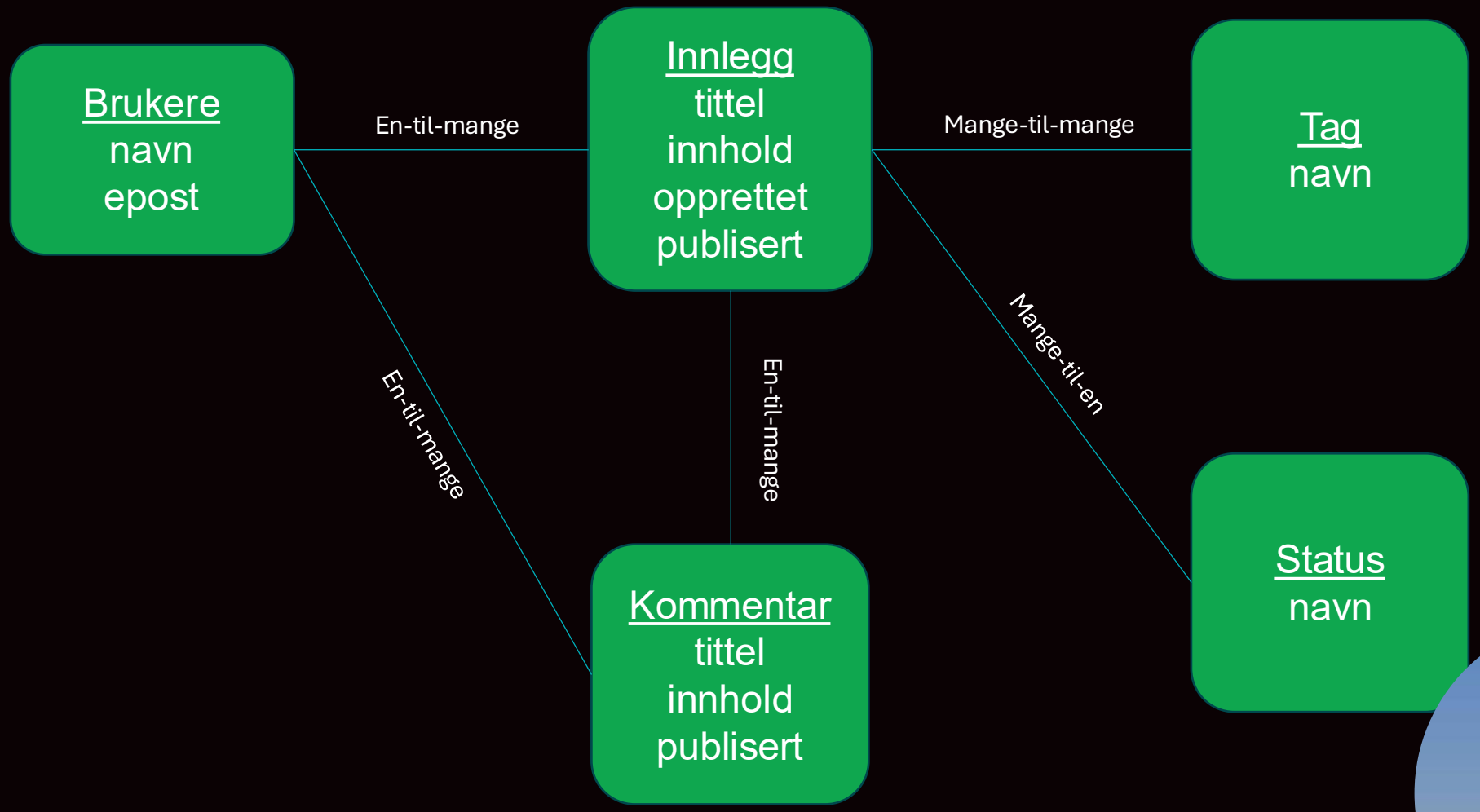
Regel 5:

Hvordan du modellerer dataene dine, avhenger – helt – av hvordan applikasjonen vil aksessere dine data.

Det vil si:

- Hvordan kommer data inn? Hyppighet? Størrelsen på data/mengde data?
- Hvordan leses data? Hvilke spørringer?

Høgskulen på Vestlandet



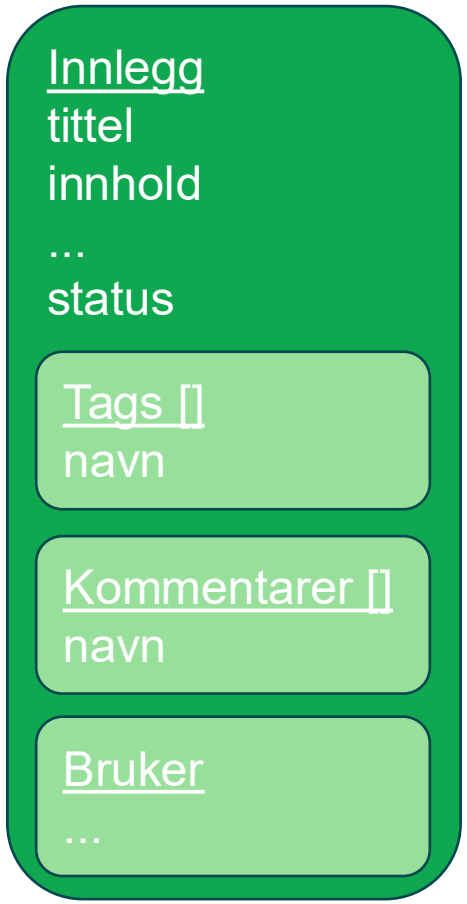
mongoDB

Høgskulen på Vestlandet

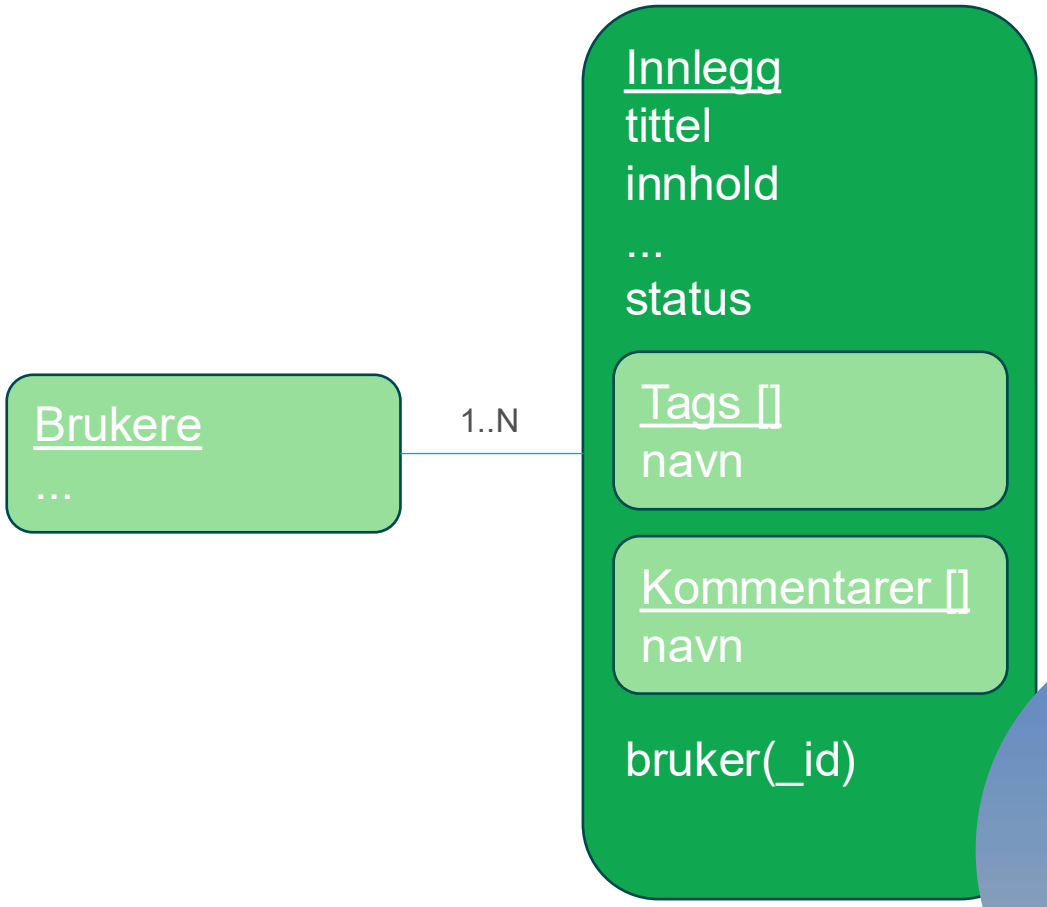
Hvordan bruker vi data?

Schema Design: Kartlegging av spørringer

Høgskulen på Vestlandet



Spørringer på **Innlegg**



Spørringer på **Innlegg og Brukere**

«Linking» eller «Embedding»?

Skjema design: Hvordan avgjør vi dette?

- ✓ Hvor ofte blir de relaterte data aksessert?
- ✓ Blir data filtrert via de relaterte data?
- ✓ Hvor ofte endrer de relaterte data seg?
- ✓ Hva er mest hensiktsmessig for din applikasjon?

Høgskulen på Vestlandet

mongoDB

Patterns

En introduksjon

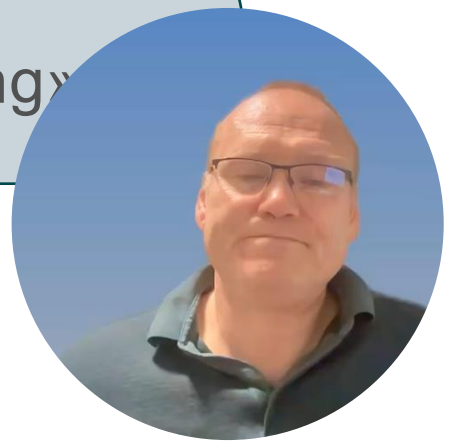
Høgskulen på Vestlandet

Læringsmål

Vi fortsetter med **skjemadesign & patterns**

Hva er et «**pattern**»?

Pattern: «Skjema Versjonering» & «Dokument Versjonering»



Litteratur

NoSQL

- ✓ Foiler
- ✓ https://www.mongodb.com/company/blog/building-with-patterns-a-summary
  - Schema Versioning
  - Document Versioning
  - Bucket

Høgskulen på Vestlandet

Hva er et «Pattern»?

Design Patterns

- ✓ «Løsninger» på vanlige problemer innen programvaredesign.
- ✓ Forhåndslagde «tegninger» eller «maler» som du kan tilpasse for å løse et tilbakevendende designproblem i koden din.
- ✓ Et «pattern» er ikke et bestemt stykke kode, men et generelt konsept for å løse et bestemt problem.

Høgskulen på Vestlandet

Skjema Versjonering

Pattern

Høgskulen på Vestlandet

Pattern: «Skjema versjonering»

Design Patterns

Migrerings Batch

Ansatte

```
{
  _id: 1,
  fornavn: "Ola", etternavn: Hanssen,
  kontor: {rom: 401, etasje: 4, fløy: B}
}
```

Ansatte

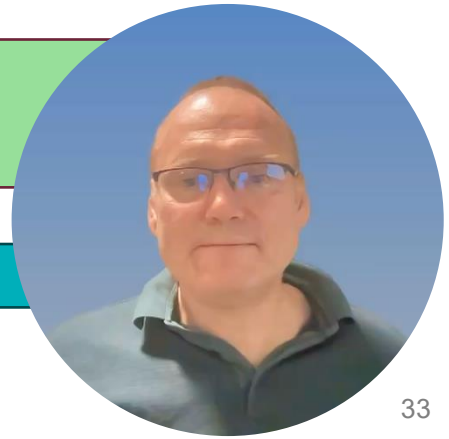
```
{
  _id: 1,
  schema_version: 2.0,
  fornavn: "Ola", etternavn: Hanssen,
  kontor: [
    {rom: 401, etasje: 4, fløy: B}
  ]
}
```

Java kode

Java kode V2

Applikasjon Versjon 1

Applikasjon Versjon 2



Pattern: «Skjema versjonering»

Design Patterns

Ansatte

```
{
   _id: 1,
   fornavn: "Ola", etternavn: Hanssen,
   kontor: {rom: 401, etasje: 4, fløy: B}
}
```

Ansatte

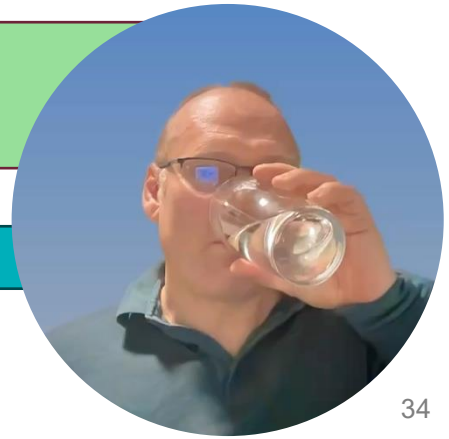
```
{
   _id: 1,
   schema_version: 2.0,
   fornavn: "Ola", etternavn: Hanssen,
   kontor: [
      {rom: 401, etasje: 4, fløy: B}
   ]
}
```

Java kode

Applikasjon Versjon 1

Java kode V2

Applikasjon Versjon 2



Pattern: «Skjema versjonering»

Design Patterns

Ansatte

```
{
  _id: 1,
  schema_version: 2.0,
  fornavn: "Ola", etternavn: Hanssen,
  kontor: [
     {rom: 401, etasje: 4, fløy: B}
  ]
}
```

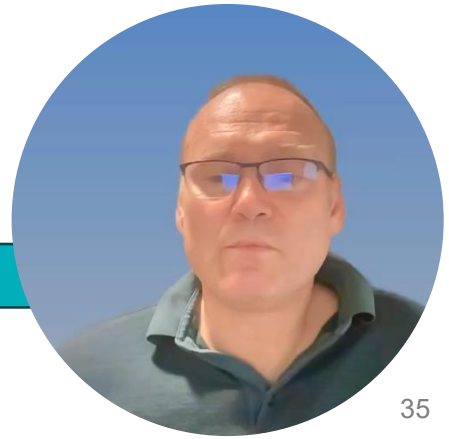
Java kode V2

Applikasjon Versjon 2

Klar for neste

ITERASJON

Applikasjon Versjon 3



Pattern: «Skjema versjonering»

Design Patterns

Fordeler	Ulemper
Ingen nedetid	Kan være behov for to indekser for «samme felt» under migrering
Kontroll ved skjema migreringer	
Reduserer fremtidig teknisk gjeld	

https://www.mongodb.com/blog/post/building-with-patterns-a-summary

Høgskulen på Vestlandet

Dokument Versjonering

Pattern

Høgskulen på Vestlandet

Pattern: «Dokument versjonering»

Design Patterns

Ansatte

```
{
   _id: 1,
   schema_version: 2.0,
   fornavn: "Ola", etternavn: Hanssen,
   kontor: {rom: 401, etasje: 4, fløy: B}
}
```

↑

Java kode V2

Hva hvis noen ønsker å vite hvor «Ola» hadde kontor i fjor?

HISTORISKE DATA

Høgskulen på Vestlandet

Pattern: «Dokument versjonering»

Design Patterns

Ansatte

```
{
   _id: 1,
   revisjon: 1 ,
   oppdatert: "2025-01-31”,
   fornavn: "Ola", etternavn: "Hanssen",
   kontor: {rom: 401, etasje: 4, fløy: B}
}
```

AnsatteHistorie

```
{
   _id: 1,
   revisjon: 1 ,
   oppdatert: "2025-01-31”,
   fornavn: "Ola", etternavn: "Hanssen",
   kontor: {rom: 401, etasje: 4, fløy: B}
}
```

Høgskulen på Vestlandet

Pattern: «Dokument versjonering»

Design Patterns

Ansatte

```
{
  _id: 1,
  revisjon: 2 ,
  oppdatert: "2026-02-15”,
  fornavn: "Ola", etternavn: "Hanssen",
  kontor: {rom: 403, etasje: 4, fløy: B}
}
```

AnsatteHistorie

```
{
  _id: 1,
  revisjon: 1 ,
  oppdatert: "2025-01-31”,
  fornavn: "Ola", etternavn: "Hanssen",
  kontor: {rom: 401, etasje: 4, fløy: B}
}
{
  _id: 1,
  revisjon: 2 ,
  oppdatert: "2026-02-15”,
  fornavn: "Ola", etternavn: "Hanssen",
  kontor: {rom: 403, etasje: 4, fløy: B}
}
```

Høgskulen på Vestlandet

«Dokument versjonering»: INSERT

Design Patterns

Ansatte

```
{
  _id: 1,
  revisjon: 1 ,
  oppdatert: "2025-01-31”,
  fornavn: "Ola", etternavn: "Hanssen",
  kontor: {rom: 401, etasje: 4, fløy: B}
}
```

AnsatteHistorie

```
{
  _id: 1,
  revisjon: 1 ,
  oppdatert: "2025-01-31”,
  fornavn: "Ola", etternavn: "Hanssen",
  kontor: {rom: 401, etasje: 4, fløy: B}
}
```

```
db.ansatte.insertOne(<JSON>)
db.ansattehistorie.insertOne(<JSON>)
```

Samme dokument i begge samlinger

Høgskulen på Vestlandet

«Dokument versjonering»: UPDATE

Design Patterns

Når en «ansatt» blir oppdatert, så må følgende skriv utføres:

- ✓ Den ansatte blir oppdatert i «ansatte» samlingen, som KUN inneholder den gjeldende revisjonen (revisjons nummer må da økes med 1).
- ✓ Den orginale informasjonen om den ansatte skrives til «ansattehistorie» samlingen.

```
db.ansatte.updateOne(
 { _id: ObjectId("680112...") },
 { $set: { kontor: {rom: 220,
                    etasje: 2, fløy: "B"}},
   $inc: { revisjon: 1 },
   $currentDate: { oppdatert: true }
 }
)
```

```
db.ansatte.aggregate( [
   { $match: { _id: ObjectId("680112...")} },
   { $set: { _id: new ObjectId() } },
   { $merge: {
       into: { db: "forelesning-03", coll: "ansattehistorie" },
       on: "_id",
       whenNotMatched: "insert"}
   }
] )
```

Høgskulen på Vestlandet

Pattern: «Dokument versjonering»

Design Patterns

Fordeler	Ulemper
Lett å implementere (selv i allerede eksisterende løsninger)	Dobbelt opp med skriv
Ingen ytelses påvirkning for spørringer mot siste versjon av data	Spørringer må adressere den riktige samlingen (Note! Ikke problematisk)

https://www.mongodb.com/blog/post/building-with-patterns-a-summary

Høgskulen på Vestlandet

Oppsummering

MongoDB Query API

Høgskulen på Vestlandet

- ✓ **Embedding** vs **Linking**
  - En-til-en: Nøkkel – Verdi, Embedding
  - En-til-få: Embedding foretrekkes
  - En-til-mange: Embedding, hvis ikke gode grunner for annet
  - En-til-VELDIG-mange: «reversjert linking»
  - Mange-til-mange: Embedding eller Linking (avhenger av de punktene over)

- ✓ **Pattern**: «Løsninger» å vanlige problemer innen programvaredesign.
- ✓ Patterns (vi så på to eksempler, finnes mange flere)
  - Schema Versioning
  - Document Versioning

DEMO